Частное образовательное учреждение высшего образования
«Санкт-Петербургский институт экономики и управления»
(СПбИЭУ)

Специальность 09.02.07 «Информационные системы и программирование»
ПМ.03 Сопровождение и обслуживание программного обеспечения компьютерных систем
МДК.03.01 Внедрение и поддержка компьютерных систем

# ОТЧЁТ

## по промежуточному контролю № 16–18

Тема 1.1. Основные методы внедрения и анализа функционирования ПО
**«CALS-технологии: цифровой след, регламент релиза, стратегия отката, тестирование и эксплуатационная документация»**
Вариант 10

Выполнил: студент группы И14-23-9
Иванов Егор Михайлович

Проверила: преподаватель
Чудинова Александра Анатольевна
Оценка: _____

Санкт-Петербург
2026

## 1. Описание задания

**Цель:** закрепить внедрение CALS-концепции на примере ИС «Умный кампус» (CampusService / .NET 8 Web API, PostgreSQL, Kubernetes): связать бизнес-требование из трекера с кодом, инфраструктурой и телеметрией, настроить правила безопасной доставки кода, спроектировать быстрый откат и миграцию БД без простоя, написать сценарии проверки после релиза и эксплуатационные документы.

**Вариант 10 — модуль «Спортивный комплекс».** По таблице вариантов для ИС «Умный кампус»:

Таблица 1 – Индивидуальный вариант

| **Блок** | **Содержание варианта 10** |
|---|---|
| 1. Цифровой след (Trace Tag / Jira) | CAMPUS-110: логирование записи в бассейн / зал |
| 3. Миграция БД (Expand & Contract) | изменение формата поля TimeSlot (string → TimeRange) |
| 4. Sanity-тест (Production) | проверка записи на слот тренировки с отменой |

По методическим указаниям выполняются четыре обязательных блока: (1) код C# — проброс тегов `jira.issue` и `campus.entity_id` в `Activity`; (2) Change Request с номером задачи, окном обслуживания и настройками Branch Protection Rules; (3) три фазы SQL-скриптов Expand → Migrate → Contract и команды отката в Kubernetes; (4) xUnit-тест для Production с обязательным шагом Cleanup, ADR в `docs/adr/`, Runbook на алерт `DatabaseConnectionPoolExhausted` и запись в `CHANGELOG.md` (Added, Fixed, Breaking Changes).

## 2. Блок 1. Единый цифровой след (Digital Trace)

Цепочка CALS для задачи: **Jira (CAMPUS-110) → ветка `feature/CAMPUS-110-timerange` → коммиты `[CAMPUS-110] …` → Pull Request → образ `sport-service:1.6.0` → теги трассы OpenTelemetry**. В контроллер `SportBookingsController` добавлена установка тегов в текущий span входящего запроса (`Activity.Current`) и в дочерний span бизнес-операции `sport.booking.create`.

Листинг 1 – CampusTelemetry.cs — имена тегов и проброс в Activity.Current

```
using System.Diagnostics;

namespace SmartCampus.Sport.Telemetry;

/// <summary>
/// Единая точка телеметрии модуля «Спортивный комплекс».
/// Имена тегов вынесены в константы, чтобы по ним можно было искать
/// трассы в Jaeger / Grafana Tempo одинаково во всех сервисах кампуса.
/// </summary>
public static class CampusTelemetry
{
    /// <summary>Имя ActivitySource — регистрируется в OpenTelemetry через
AddSource().</summary>
    public const string SourceName = "SmartCampus.Sport";

    public static readonly ActivitySource Source = new(SourceName, "1.6.0");

    /// <summary>Задача Jira, по которой внесено изменение (цифровой след
CALS).</summary>
    public const string JiraIssueTag = "jira.issue";

    /// <summary>Значение тега для варианта 10.</summary>
    public const string JiraIssue = "CAMPUS-110";

    // Доменные идентификаторы сущностей.
    // campus.entity_id — единый тег из методических указаний (одинаков во всех модулях
кампуса),
    // campus.booking_id — тот же идентификатор с доменным именем для фильтров
спорткомплекса.
    public const string EntityIdTag = "campus.entity_id";
    public const string BookingIdTag = "campus.booking_id";
    public const string FacilityIdTag = "campus.facility_id";
    public const string TimeRangeTag = "campus.time_range";
    public const string ModuleTag = "campus.module";
    public const string ModuleName = "sport-complex";

    /// <summary>
    /// Проставляет обязательные теги на текущий span (Activity.Current),
    /// который создала инструментация ASP.NET Core для входящего запроса.
    /// </summary>
    public static Activity? TagCurrent(int? facilityId, Guid? bookingId = null)
    {
        var activity = Activity.Current;
        if (activity is null)
        {
            return null; // трассировка выключена или запрос не сэмплирован
        }

        activity.SetTag(JiraIssueTag, JiraIssue);
        activity.SetTag(ModuleTag, ModuleName);
        if (facilityId is not null)
        {
            activity.SetTag(FacilityIdTag, facilityId.Value);
        }

        if (bookingId is not null)
        {
            activity.SetTag(EntityIdTag, bookingId.Value.ToString());
            activity.SetTag(BookingIdTag, bookingId.Value.ToString());
        }

        // Baggage уходит в заголовке baggage во все исходящие вызовы,
        // поэтому номер задачи увидят и нижележащие сервисы (биллинг, турникеты).
        activity.SetBaggage(JiraIssueTag, JiraIssue);
        return activity;
    }
}
```

Листинг 2 – SportBookingsController.cs — цифровой след при записи на слот

```
    /// <summary>Создать бронь зала.</summary>
    [HttpPost]
    [ProducesResponseType<BookingResponse>(StatusCodes.Status201Created)]
```

```
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<ActionResult<BookingResponse>> Create(CreateBookingRequest request,
CancellationToken ct)
    {
        // 1. Цифровой след: span входящего HTTP-запроса получает тег задачи Jira
        //    и доменные идентификаторы. По тегу jira.issue = CAMPUS-110 в Jaeger
        //    находятся все запросы, прошедшие через изменённый код.
        var activity = CampusTelemetry.TagCurrent(request.FacilityId);

        // 2. Дочерний span бизнес-операции — видно, сколько времени заняла
        //    именно логика бронирования, а не сеть и сериализация.
        using var operation =
CampusTelemetry.Source.StartActivity("sport.booking.create");
        operation?.SetTag(CampusTelemetry.JiraIssueTag, CampusTelemetry.JiraIssue);
        operation?.SetTag(CampusTelemetry.FacilityIdTag, request.FacilityId);
```

Листинг 3 – SportBookingsController.cs — тег идентификатора сущности

```
        // Идентификатор сущности известен до записи — сразу кладём его в трассу,
        // чтобы при ошибке БД было видно, какая бронь не сохранилась.
        activity?.SetTag(CampusTelemetry.EntityIdTag, booking.Id.ToString());
        activity?.SetTag(CampusTelemetry.BookingIdTag, booking.Id.ToString());
        operation?.SetTag(CampusTelemetry.EntityIdTag, booking.Id.ToString());
        operation?.SetTag(CampusTelemetry.BookingIdTag, booking.Id.ToString());
```

**Связь лога в консоли контейнера с графом вызовов в Jaeger**

1) Клиент или API-шлюз передаёт заголовок W3C `traceparent: 00-<trace-id 32 hex>-<span-id 16 hex>-01`; если его нет, ASP.NET Core создаёт новый trace-id.
2) Инструментация OpenTelemetry продолжает трассу: все span'ы запроса (HTTP, `sport.booking.create`, SQL) получают тот же trace-id и экспортируются по OTLP (порт 4317) в Jaeger.
3) Логгер пишет trace-id в каждую строку лога (scope логирования `IncludeScopes = true`), и тот же trace-id API возвращает клиенту в заголовке `X-Trace-Id`.
4) Дежурный берёт trace-id из строки `kubectl logs deploy/sport-service -n production | grep CAMPUS-110` и открывает в Jaeger UI (порт 16686): Search → Trace ID. Jaeger показывает граф (waterfall) запроса с временем каждого этапа.
5) Обратный поиск: Jaeger → сервис smart-campus-sport → Tags `jira.issue=CAMPUS-110` — все запросы, прошедшие через изменённый по задаче код.

Таблица 2 – Зафиксированная сквозная цепочка

| Jira | Git Commit | OpenTelemetry Trace Tag |
|---|---|---|

| | | |
|---|---|---|
| CAMPUS-110 | [CAMPUS-110] Add tracing tags and business logic<br>(ветка feature/CAMPUS-110-timerange) | jira.issue = CAMPUS-110,<br>campus.entity_id = <id брони>;<br>trace-id лога = X-Trace-Id = Trace ID в Jaeger |




Рисунок 1 – Запущенный на ПК сервис v1.6.0: запись на слот в новом формате timeRange, поле timeSlot для старых клиентов и traceId запроса (цифровой след)

Листинг 4 – История Git репозитория (git log --graph): ветка feature/CAMPUS-110-timerange и коммиты с префиксом [CAMPUS-110]

```
* 33a6c66 25.10.2026 11:58 [CAMPUS-110] Validate k8s manifests in CI, update CR-2026-110
rollback plan and docs
* 26391dc 25.10.2026 11:58 [CAMPUS-110] Add Kubernetes Deployment manifest v1.6.0 and rollback
manifest v1.5.3
* 9b69b01 25.10.2026 04:21 [CAMPUS-110] Update OpenTelemetry packages to 1.20.1 (vulnerability
fix)
* 948aa07 25.10.2026 03:24 [CAMPUS-110] CHANGELOG 1.6.0: entity_id tag, stand tests, Runbook
* 6d2190c 25.10.2026 03:21 README: add PK 15-20
* 4637987 25.10.2026 03:21 pk-20: ARM monitor with Serilog dual sinks, latency metric, meal
statement by benefit categories
* 4d1dcce 25.10.2026 03:21 pk-19: meal daily report generator (CSV by category, SHA-256,
retries, cache)
*   9ebd5a5 25.10.2026 03:21 [CAMPUS-110] Merge feature/CAMPUS-110-timerange: PK 16-18 release
v1.6.0
|\
| * f4d934a 25.10.2026 03:21 [CAMPUS-110] Add CR v1.6.0, branch protection, Runbook, ADR,
CHANGELOG and CI checks
| * 36058eb 25.10.2026 03:21 [CAMPUS-110] Add production sanity test with cleanup and stand
tests
| * dbcd0ba 25.10.2026 03:21 [CAMPUS-110] Add Expand and Contract migrations for TimeSlot ->
TimeRange
| * c0c52dd 25.10.2026 03:21 [CAMPUS-110] Add tracing tags and business logic
|/
* 47429a5 25.10.2026 03:21 pk-15: OrderService release regulation, smoke tests, rollback plan,
Runbooks and ADR (variant 10, WAF before API)
* 7236735 23.10.2026 01:45 Add Visual Studio launch profiles for test scenarios
```

# 3. Блок 2. Регламент релиза и Change Management

## 3.1. Branch Protection Rules для ветки main

Таблица 3 – Правила защиты ветки

| **Требование** | **Настройка (GitHub → Settings → Branches → main)** |
|---|---|
| Запрет прямого пуша | Require a pull request before merging + Do not allow bypassing (и для администраторов) |
| Force Push is disabled | Allow force pushes — выключено, Allow deletions — выключено |
| Обязательные CI/CD-тесты (xUnit) | Require status checks to pass: xunit-tests, db-migrations; Require branches to be up to date |
| 2 одобрения коллег + 1 Tech Lead | Required approvals = 3 + Require review from Code Owners (CODEOWNERS: * @campus/tech-leads) |

| | |
|---|---|
| Одобрение актуального кода | Dismiss stale approvals, Require approval of the most recent push |
| Формат коммитов | Ruleset: сообщение по шаблону ^\[CAMPUS-\d+\] .+ |

Значение 3 выбрано потому, что GitHub засчитывает одобрение Code Owner в общее число: при обязательном ревью Code Owner в Pull Request гарантированно будет 2 одобрения коллег и 1 одобрение Tech Lead.

### 3.2. Change Request релиза v1.6.0

Таблица 4 – Карточка Change Request (docs/change-requests/CR-2026-110.md)

| **Параметр** | **Значение** |
|---|---|
| Номер CR | CR-2026-110 |
| Система | ИС «Умный кампус», sport-service (модуль «Спортивный комплекс») v1.6.0 (на проде — v1.5.3) |
| Задача Jira / связанные задачи | CAMPUS-110 (этот релиз); CAMPUS-118 — фаза Contract, следующий релиз |
| Описание изменений | слот бронирования TimeSlot (строка) → диапазон sport.timerange; API принимает оба формата; теги OpenTelemetry; sanity-тест с очисткой |
| Окно обслуживания | среда, 28.10.2026, 02:00–04:00 МСК — минимум трафика (менее 1 запроса/мин), сразу после бэкапа 01:30, не пятница |
| План работ | 1) Expand (01_expand.sql); 2) выкатка подов v1.6.0; 3) Sanity-тест; 4) проверка следа в Jaeger; 5) Migrate (02_migrate.sql) |
| План отката | kubectl rollout undo deployment/sport-service -n production |
| Защита ветки | PR обязателен, force push запрещён, xunit-tests + db-migrations + k8s-manifests, 2 коллеги + Tech Lead |

### 3.3. Pull Request и Quality Gates в CI

Изменение доставляется в `main` только через Pull Request, как требует регламент. Pull Request №1 `[CAMPUS-110] Kubernetes manifests and rollback manifest for sport-service v1.6.0` (ветка `feature/CAMPUS-110-k8s-manifests` → `main`) содержит описание изменений, ссылку на CR-2026-110, план отката и чек-лист ревью: 2 коллеги и Tech Lead. Для этого PR GitHub Actions автоматически запустил три обязательные проверки: `xunit-tests` (сборка и xUnit-тесты), `db-migrations` (Expand → Migrate → Contract на PostgreSQL 16) и `k8s-manifests` (проверка YAML-манифестов по схеме Kubernetes 1.30).

Ссылка: https://github.com/Egorvan606-senior/mdk03/pull/1

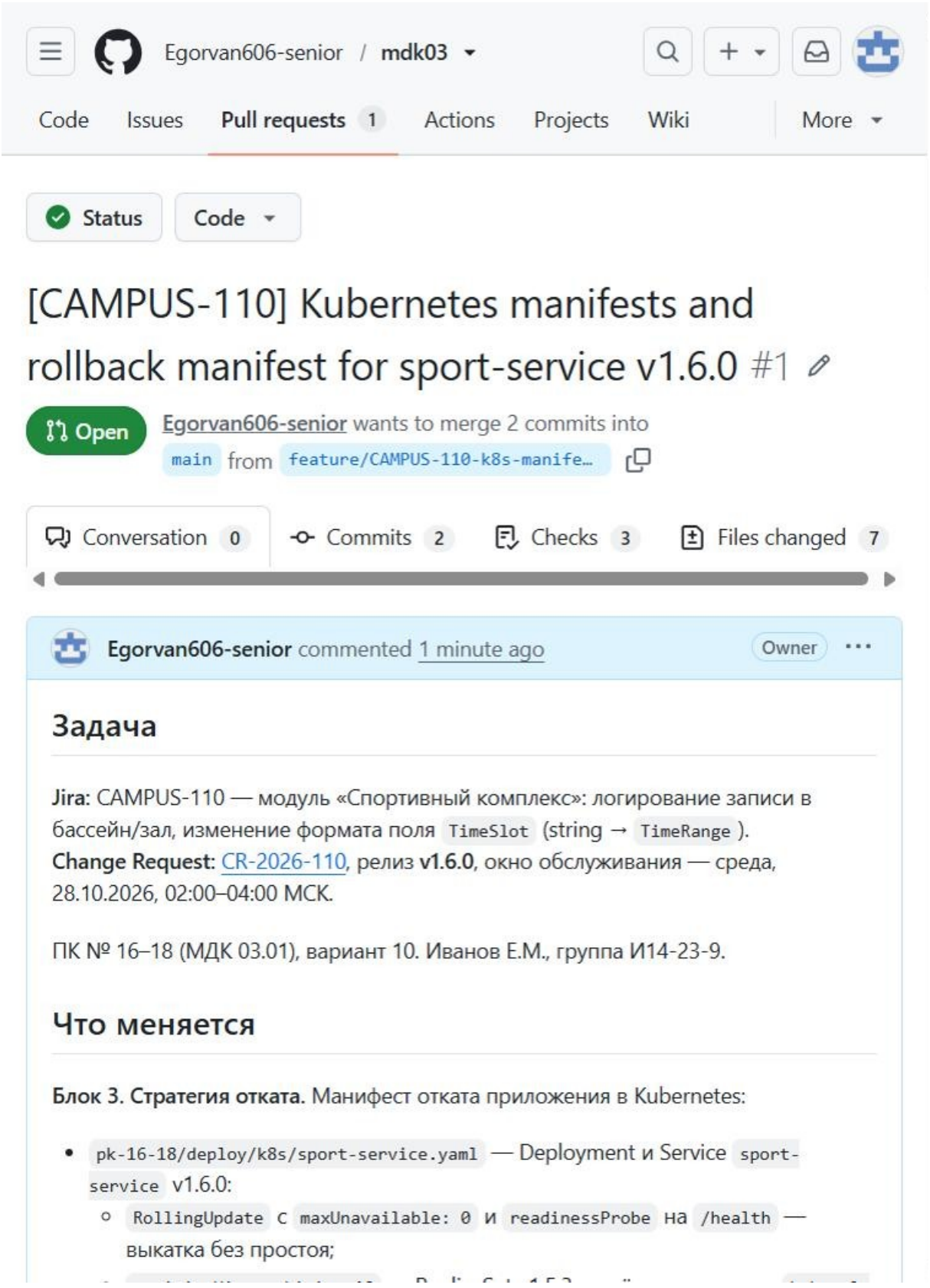



Рисунок 2 – Pull Request №1 [CAMPUS-110] на GitHub: описание изменений со ссылкой на Change Request и план отката

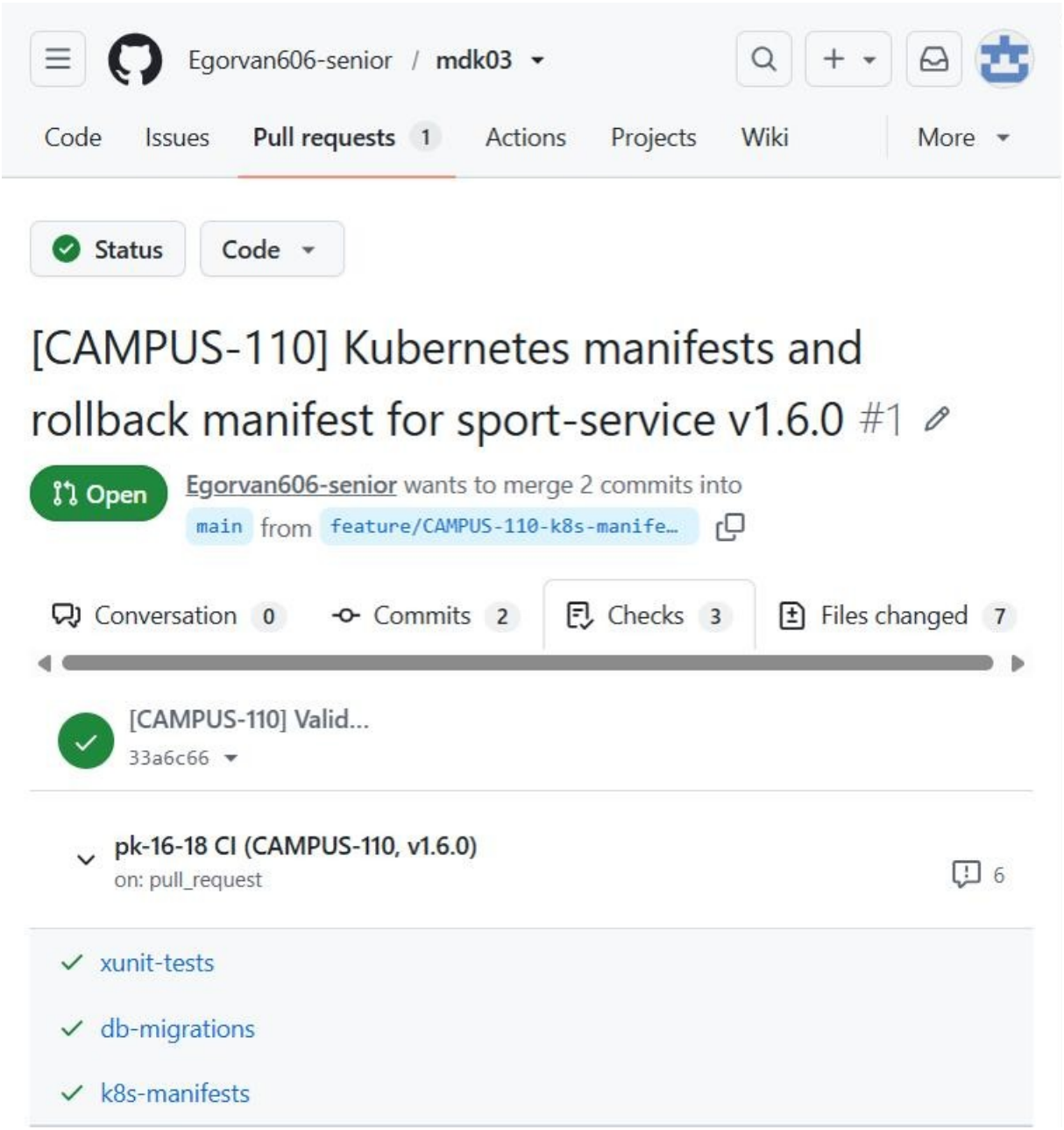



Рисунок 3 – Обязательные проверки Pull Request (GitHub Actions): xunit-tests, db-migrations, k8s-manifests пройдены

# 4. Блок 3. Стратегия отката и миграция БД (Expand and Contract)

## 4.1. Откат деплоя в Kubernetes при росте ошибок 5xx

Листинг 5 – Команды отката (CR-2026-110, раздел 5)

```
# История ревизий — убедиться, что есть предыдущая стабильная (v1.5.3)
kubectl rollout history deployment/sport-service -n production

# Экстренный возврат к предыдущей стабильной версии (≈ 30 секунд)
kubectl rollout undo deployment/sport-service -n production

# Контроль завершения отката: все поды Ready
kubectl rollout status deployment/sport-service -n production
```

```
# Проверка, что работает нужный образ
kubectl get deployment sport-service -n production -o
jsonpath='{.spec.template.spec.containers[0].image}'
```

Чтобы `kubectl rollout undo` гарантированно работал, подготовлен манифест релиза `deploy/k8s/sport-service.yaml`. В нём хранится история ревизий (`revisionHistoryLimit: 10`, ReplicaSet v1.5.3 остаётся в кластере). Выкатка идёт без простоя: `maxUnavailable: 0` и readiness-проба `/health`. Зависшая выкатка через 180 секунд завершается ошибкой (`progressDeadlineSeconds`). Номер CR и задачи записан в аннотацию `kubernetes.io/change-cause` и виден в `kubectl rollout history`.

Листинг 6 – deploy/k8s/sport-service.yaml — Deployment v1.6.0 (фрагмент)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sport-service
  namespace: production
  labels:
    app.kubernetes.io/name: sport-service
    app.kubernetes.io/part-of: smart-campus
    app.kubernetes.io/version: "1.6.0"
  annotations:
    kubernetes.io/change-cause: "CR-2026-110 [CAMPUS-110] sport-service 1.6.0: TimeSlot ->
TimeRange, OpenTelemetry tags"
    campus.university.edu/jira-issue: "CAMPUS-110"
    campus.university.edu/change-request: "CR-2026-110"
spec:
  replicas: 3
  revisionHistoryLimit: 10
  progressDeadlineSeconds: 180
  minReadySeconds: 10
  selector:
    matchLabels:
      app.kubernetes.io/name: sport-service
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Резервный способ — декларативный манифест отката `deploy/k8s/rollback/sport-service-v1.5.3.yaml`, Deployment с образом предыдущей стабильной версии. Он нужен, если история ревизий потеряна или откат выполняется через GitOps: `kubectl apply -f deploy/k8s/rollback/sport-service-v1.5.3.yaml`. Оба манифеста проверяются в CI (`kubernetes-validate --strict`, Kubernetes 1.30).

Листинг 7 – deploy/k8s/rollback/sport-service-v1.5.3.yaml — манифест отката (начало)

```
# Манифест отката sport-service на предыдущую стабильную версию v1.5.3
# Change Request: CR-2026-110, раздел 5 «План отката». Задача Jira: CAMPUS-110
#
# Основной способ отката — команда (≈ 30 секунд, ReplicaSet v1.5.3 уже есть в кластере):
#   kubectl rollout undo deployment/sport-service -n production
#   kubectl rollout status deployment/sport-service -n production
#
```

```
# Этот манифест — резервный (декларативный) способ. Применяется, если:
#   * история ревизий потеряна (Deployment пересоздан, revisionHistoryLimit исчерпан);
#   * откат выполняется через GitOps: манифест коммитится в main и применяется CD;
#   * `rollout undo` вернул не ту ревизию (проверка: kubectl rollout history ...).
#
#   kubectl apply -f deploy/k8s/rollback/sport-service-v1.5.3.yaml
#   kubectl rollout status deployment/sport-service -n production --timeout=10m
#   kubectl get deployment sport-service -n production \
#     -o jsonpath='{.spec.template.spec.containers[0].image}'   # ожидаем ...:1.5.3
#
# Откат БД НЕ нужен: после фазы Expand v1.5.3 работает со старой колонкой time_slot,
# а триггер синхронизации заполняет time_range. После фазы Contract (CR-2026-118)
# этот манифест применять нельзя — колонки time_slot уже нет.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sport-service
  namespace: production
  labels:
    app.kubernetes.io/name: sport-service
    app.kubernetes.io/part-of: smart-campus
    app.kubernetes.io/version: "1.5.3"
  annotations:
    kubernetes.io/change-cause: "ROLLBACK CR-2026-110 [CAMPUS-110]: sport-service 1.6.0 ->
1.5.3 (5xx > 3% or failed sanity test)"
```

Откат БД при этом не нужен: после фазы Expand версия v1.5.3 продолжает работать со старой колонкой `time_slot`, а триггер совместимости заполняет новую. В этом главное преимущество паттерна.

### 4.2. Три фазы изменения TimeSlot (string) → TimeRange

Таблица 5 – Expand and Contract

| Фаза | Скрипт | Что происходит | Совместимость |
|---|---|---|---|
| 1. Expand | 01_expand.sql | тип sport.timerange, nullable-колонка time_range, триггер синхронизации | v1.5.x и v1.6.0 работают одновременно |
| 2. Migrate | 02_migrate.sql | пакетный UPDATE исторических записей, журнал ошибок разбора | сервис пишет в обе колонки (dual write) |
| 3. Contract | 03_contract.sql | NOT NULL, удаление time_slot и триггера, EXCLUDE против пересечений | только v1.6.x+, отдельный CR |

Листинг 8 – 01_expand.sql — фаза Expand (фрагмент)

```
SET lock_timeout = '3s';
SET statement_timeout = '60s';

BEGIN;

-- 1. Диапазонный тип: интервал времени [начало, конец)
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1
                   FROM pg_type t
                   JOIN pg_namespace n ON n.oid = t.typnamespace
                   WHERE n.nspname = 'sport' AND t.typname = 'timerange') THEN
        CREATE TYPE sport.timerange AS RANGE (subtype = time);
    END IF;
END $$;

-- 2. Новая колонка (NULL допустим до окончания фазы 2)
ALTER TABLE sport.bookings
    ADD COLUMN IF NOT EXISTS time_range sport.timerange NULL;

COMMENT ON COLUMN sport.bookings.time_slot  IS 'DEPRECATED (CAMPUS-110): заменена на
time_range, удаляется в 03_contract.sql';
COMMENT ON COLUMN sport.bookings.time_range IS 'CAMPUS-110: слот бронирования, диапазон
```

```
[начало, конец)';
```

Листинг 9 – 02_migrate.sql — фаза Migrate: перелив данных пакетами

```
CREATE OR REPLACE PROCEDURE sport.backfill_time_range(p_batch_size integer DEFAULT 1000)
LANGUAGE plpgsql
AS $$
DECLARE
    v_ids       uuid[];
    v_batch     integer := 0;
    v_updated   bigint  := 0;
    v_failed    bigint  := 0;
    v_rows      integer;
BEGIN
    LOOP
        -- следующая пачка необработанных строк (ошибочные пропускаем)
        SELECT array_agg(id)
        INTO v_ids
        FROM (
            SELECT b.id
            FROM sport.bookings b
            WHERE b.time_range IS NULL
              AND NOT EXISTS (SELECT 1 FROM sport.bookings_migration_errors e
                              WHERE e.booking_id = b.id)
            ORDER BY b.id
            LIMIT p_batch_size
            FOR UPDATE SKIP LOCKED      -- не ждём строки, которые сейчас правит
приложение
        ) s;

        EXIT WHEN v_ids IS NULL;

        UPDATE sport.bookings
        SET time_range = sport.parse_time_slot(time_slot)
        WHERE id = ANY (v_ids)
          AND time_range IS NULL;
        GET DIAGNOSTICS v_rows = ROW_COUNT;

        -- то, что не разобралось, — в журнал ошибок
        INSERT INTO sport.bookings_migration_errors (booking_id, time_slot, reason)
        SELECT id, time_slot, 'Некорректный формат TimeSlot'
        FROM sport.bookings
        WHERE id = ANY (v_ids) AND time_range IS NULL
        ON CONFLICT (booking_id) DO NOTHING;

        v_failed  := v_failed + (SELECT count(*) FROM sport.bookings
                                 WHERE id = ANY (v_ids) AND time_range IS NULL);
        v_updated := v_updated + v_rows;
        v_batch   := v_batch + 1;

        COMMIT;   -- фиксируем пачку, освобождаем блокировки

        RAISE NOTICE 'CAMPUS-110 backfill: пачка %, обработано %, перенесено всего %,
ошибок %',
            v_batch, coalesce(array_length(v_ids, 1), 0), v_updated - v_failed, v_failed;

        PERFORM pg_sleep(0.05);  -- даём «подышать» рабочей нагрузке
    END LOOP;

    RAISE NOTICE 'CAMPUS-110 backfill завершён: пачек %, ошибок %', v_batch, v_failed;
END $$;

CALL sport.backfill_time_range(1000);
```

Листинг 10 – 03_contract.sql — фаза Contract (фрагмент)

```
-- 1. NOT NULL без длительной блокировки:
--    CHECK ... NOT VALID ставится мгновенно, VALIDATE сканирует таблицу под
--    мягкой блокировкой (чтение и запись продолжают работать),
--    а SET NOT NULL (PostgreSQL 12+) использует проверенный CHECK и не сканирует заново.
ALTER TABLE sport.bookings
    ADD CONSTRAINT ck_bookings_time_range_not_null CHECK (time_range IS NOT NULL) NOT
VALID;
ALTER TABLE sport.bookings
```

```
    VALIDATE CONSTRAINT ck_bookings_time_range_not_null;

BEGIN;

ALTER TABLE sport.bookings ALTER COLUMN time_range SET NOT NULL;
ALTER TABLE sport.bookings DROP CONSTRAINT ck_bookings_time_range_not_null;

-- 2. Убираем механизмы переходного периода
DROP TRIGGER IF EXISTS bookings_sync_time_range ON sport.bookings;
DROP FUNCTION IF EXISTS sport.trg_bookings_sync_time_range();

-- 3. Удаляем старую структуру
ALTER TABLE sport.bookings DROP COLUMN IF EXISTS time_slot;
DROP FUNCTION IF EXISTS sport.format_time_slot(sport.timerange);
-- parse_time_slot оставляем: используется при импорте старых выгрузок

-- 4. Бизнес-правило, ради которого всё делалось: БД сама запрещает
--    пересекающиеся активные брони одного зала в один день.
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE sport.bookings
    ADD CONSTRAINT ck_bookings_time_range_valid
        CHECK (NOT isempty(time_range) AND lower_inc(time_range) AND NOT
upper_inc(time_range));

ALTER TABLE sport.bookings
    ADD CONSTRAINT ex_bookings_no_overlap
        EXCLUDE USING gist (facility_id WITH =, booking_date WITH =, time_range WITH &&)
        WHERE (status = 'Active');

COMMENT ON COLUMN sport.bookings.time_range IS 'Слот бронирования [начало, конец); без
пересечений для активных броней';

COMMIT;

INSERT INTO public.schema_changelog (jira_issue, phase, script)
VALUES ('CAMPUS-110', 'contract', '03_contract.sql');
```

Скрипты проверены сквозным тестом `db/test/run_migration_test.sh` на PostgreSQL 16.15. Проверено: повторный запуск Expand (идемпотентность), работа триггера для старого клиента, откат Expand, перенос 4966 записей с журналом 35 некорректных, отказ Contract при неразобранных ошибках, Contract и запрет пересечения броней. Протокол прогона — `docs/evidence/migration-test.txt`.

## 5. Блок 4. Тестирование и эксплуатационная документация

### 5.1. Sanity-тест на Production: запись на слот тренировки с отменой

Тест выполняется сразу после деплоя (`dotnet test --filter Category=Sanity`, адрес и токен — из секретов CI). Запись создаётся в служебном зале 999 на дату через 60 дней и помечается `sanity-bot`. Шаг

Cleanup выполняется в `DisposeAsync` всегда, даже при упавших проверках, поэтому на боевой базе не остаётся тестовых броней.

Листинг 11 – ProductionSanityTests.cs — сценарий и проверки

```
    [SanityFact]
    public async Task CreateBooking_InNewTimeRangeFormat_IsStoredAndReturnedWithLegacyField()
    {
        // Arrange — тестовая бронь 22:00–22:30 в служебном зале
        var date = DateOnly.FromDateTime(DateTime.Today.AddDays(60));
        var request = new
        {
            facilityId = SanityFacilityId,
            studentId = "SANITY-BOT",
            bookingDate = date,
            timeRange = new { start = "22:00", end = "22:30" }
        };

        // Act 1 — создание (боевой сценарий)
        using var created = await _http.PostAsJsonAsync(Endpoint, request, Json);
        string createdBody = await created.Content.ReadAsStringAsync();
        Assert.True(created.StatusCode == HttpStatusCode.Created,
            $"Ожидался 201 Created, получено {(int)created.StatusCode}: {createdBody}");

        var booking = JsonSerializer.Deserialize<BookingDto>(createdBody, Json)!;
        _createdIds.Add(booking.Id); // регистрируем для Cleanup ДО остальных проверок

        // Assert — новый формат и обратная совместимость
        Assert.Equal("22:00", booking.TimeRange.Start);
        Assert.Equal("22:30", booking.TimeRange.End);
        Assert.Equal("22:00-22:30", booking.TimeSlot);   // старое поле для клиентов v1.5
        Assert.Equal(30, booking.DurationMinutes);
        Assert.Equal("Active", booking.Status);
        Assert.True(created.Headers.Contains("X-Trace-Id"), "Нет заголовка X-Trace-Id —
цифровой след не формируется");
        Assert.False(string.IsNullOrWhiteSpace(booking.TraceId));

        // Act 2 — чтение записи из БД через API
        var loaded = await _http.GetFromJsonAsync<BookingDto>($"{Endpoint}/{booking.Id}",
Json);
        Assert.NotNull(loaded);
        Assert.Equal(booking.Id, loaded!.Id);
        Assert.Equal(booking.TimeRange, loaded.TimeRange);

        // Act 3 — повторная бронь того же слота должна быть отклонена (запрет пересечений)
        using var duplicate = await _http.PostAsJsonAsync(Endpoint, request, Json);
        Assert.Equal(HttpStatusCode.Conflict, duplicate.StatusCode);
    }

    [SanityFact]
```

Листинг 12 – ProductionSanityTests.cs — обязательный шаг Cleanup

```
    /// <summary>
    /// CLEANUP — мгновенная очистка тестовых данных. Вызывается xUnit после
    /// каждого теста независимо от результата, поэтому на проде не остаётся
    /// «мусорных» броней, а служебный слот освобождается для следующего запуска.
    /// </summary>
    public async Task DisposeAsync()
    {
        var errors = new List<string>();
        foreach (var id in _createdIds)
        {
            try
            {
                using var deleted = await _http.DeleteAsync($"{Endpoint}/{id}");
                if (deleted.StatusCode != HttpStatusCode.NoContent)
                {
                    errors.Add($"{id}: DELETE вернул {(int)deleted.StatusCode}");
                    continue;
                }

                // контроль: запись действительно удалена
                using var check = await _http.GetAsync($"{Endpoint}/{id}");
                if (check.StatusCode != HttpStatusCode.NotFound)
                {
```

```
                    errors.Add($"{id}: после очистки GET вернул {(int)check.StatusCode}");
                }
            }
            catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
            {
                errors.Add($"{id}: {ex.Message}");
            }
        }

        _http.Dispose();

        if (errors.Count > 0)
        {
            // Падение на очистке — сигнал дежурному удалить записи вручную
            throw new InvalidOperationException("Sanity Cleanup не выполнен: " + string.Join("
", errors));
        }
    }
```

Тот же сценарий без PostgreSQL и сети проверяется стенд-тестами `StandSanityTests` (WebApplicationFactory): их можно запускать на каждом Pull Request. Тест `DigitalTraceTests` проверяет теги цифрового следа и совпадение `X-Trace-Id` с TraceId спана. При запуске на ПК пакеты OpenTelemetry обновлены с 1.9.0 до 1.20.1 — Visual Studio сообщила об уязвимости в версии 1.9.0; после обновления решение собирается и все тесты проходят.

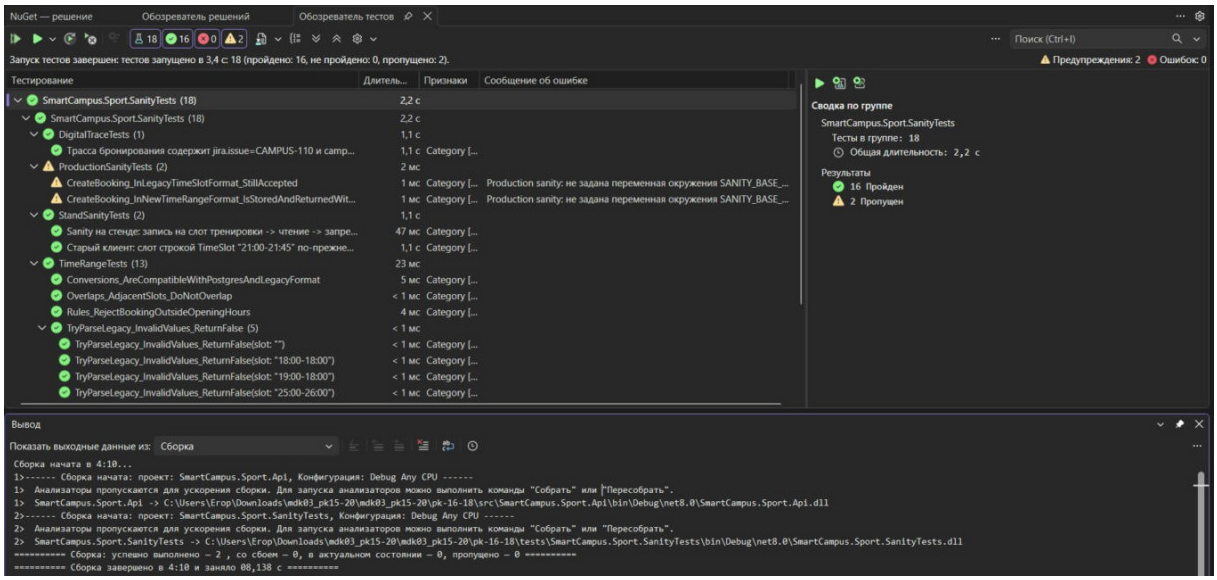



Рисунок 4 – Обозреватель тестов Visual Studio: модульные, стенд- и trace-тесты пройдены, production sanity пропущен без SANITY_BASE_URL

## 5.2. Runbook на алерт DatabaseConnectionPoolExhausted

Листинг 13 – docs/runbooks/RB-001-database-connection-pool-exhausted.md

```
# Runbook RB-001. Алерт `DatabaseConnectionPoolExhausted`

**Сервис:** `sport-service` v1.6.0 (модуль «Спортивный комплекс») · **Кому:** дежурный инженер
(SRE / L2)
**Условие алерта:** занято > 95 % соединений пула Npgsql (`Maximum Pool Size = 100`) дольше 2
минут
**Симптом для пользователей:** запись на тренировку «зависает», API отвечает 500 / 504, в логе
```

```
`The connection pool has been exhausted`

## Шаг 1. Диагностика (1–2 минуты)

```bash
# 1. Сколько подов и не перезапускаются ли они
kubectl get pods -n production -l app=sport-service

# 2. Ошибки пула в логах за 10 минут
kubectl logs -n production deploy/sport-service --since=10m | grep -i "connection pool"

# 3. Кто держит соединения в PostgreSQL (DBA / доступ только на чтение)
psql -h db.campus.local -U sre_readonly -d campus -c \
  "SELECT state, count(*) FROM pg_stat_activity WHERE application_name LIKE 'sport-service%'
GROUP BY state;"
```

* Много соединений в состоянии `idle in transaction` / долгие запросы — «залип» фоновый воркер
→ **Шаг 2**.
* PostgreSQL недоступен целиком — это инцидент БД → **сразу Шаг 3 (эскалация DBA)**.

## Шаг 2. Действие — перезапуск воркера

Перезапуск подов сервиса освобождает все соединения пула (RollingUpdate — без простоя):

```bash
kubectl rollout restart deployment/sport-service -n production
kubectl rollout status  deployment/sport-service -n production --timeout=5m
```

Через 2 минуты проверьте: алерт погас, sanity-проверка записи на слот проходит
(`dotnet test --filter Category=Sanity`), доля 5xx < 1 %.

## Шаг 3. Критерий перехода к откату

**Откатывайте релиз, если выполняется ЛЮБОЕ условие:**

1. алерт повторился **в течение 15 минут** после перезапуска;
2. доля ответов **5xx > 2 % дольше 5 минут** после перезапуска;
3. проблема началась **в течение 24 часов после выкатки v1.6.0** (вероятная причина — новый код
dual write).

```bash
kubectl rollout undo   deployment/sport-service -n production
kubectl rollout status deployment/sport-service -n production
```

Откат БД не нужен: фаза Expand обратно совместима, v1.5.3 работает со старой колонкой
`time_slot`.
После отката — сообщение в `#campus-ops`, инцидент в Service Desk, разбор (Post-Mortem) в
течение 2 рабочих дней.

## Эскалация

Если через 10 минут после шагов 2–3 сервис не восстановлен — дежурный DBA и Tech Lead модуля в
`#emergency-sre`.
Приложите: время начала, вывод команд шага 1, TraceId одного неуспешного запроса.
```

## 5.3. ADR и CHANGELOG

Листинг 14 – docs/adr/0009-sport-timeslot-to-timerange.md (начало)

```
# ADR-0009. Хранение слота бронирования спорткомплекса как диапазона `timerange` вместо строки
`TimeSlot`

* **Статус:** Принято (Accepted)
* **Дата:** 2026-10-23
* **Автор:** Иванов Е.М. (группа И14-23-9)
* **Задача:** CAMPUS-110
* **Связанные документы:** [CR-2026-110](../change-requests/CR-2026-110.md), [CHANGELOG]
(../../CHANGELOG.md)
* **Заменяет / заменено:** —

## Контекст

В модуле «Спортивный комплекс» слот бронирования зала хранится в колонке
`sport.bookings.time_slot` типа `varchar(20)` в виде строки `"18:00-19:30"`.
```

```
Проблемы текущего решения:

1. **Нет контроля пересечений.** Сравнить две строки как интервалы БД не может.
   Проверка выполняется только в коде сервиса и не защищена от гонки двух
   одновременных запросов — отсюда двойные бронирования одного зала.
2. **Грязные данные.** Разные клиенты (киоск, мобильное приложение, импорт из Excel)
   пишут разные форматы: `18.00 - 19.30`, `9:00-10:00`, встречаются некорректные
   значения (`весь день`, конец раньше начала).
```

Листинг 15 – CHANGELOG.md — запись о релизе 1.6.0 (Keep a Changelog)

```
## [1.6.0] - 2026-10-28

### Added
- [CAMPUS-110] Тип `sport.timerange` и колонка `sport.bookings.time_range` —
  слот бронирования хранится как диапазон времени `[начало, конец)`.
- [CAMPUS-110] Поле `timeRange { start, end }` и `durationMinutes` в API
  `POST/GET /api/v1/sport/bookings`; запрос принимает новый формат слота.
- [CAMPUS-110] Трассировка OpenTelemetry: теги `jira.issue`, `campus.entity_id`,
`campus.booking_id`,
  `campus.facility_id`, `campus.time_range`; заголовок ответа `X-Trace-Id`; TraceId в каждой
строке лога.
- [CAMPUS-110] Двойная запись (dual write) в `time_slot` и `time_range`,
  режим переключается настройкой `BookingSchema:Phase`.
- [CAMPUS-110] Миграции Expand and Contract: `01_expand.sql`, `02_migrate.sql`,
  `03_contract.sql`, скрипт отката `rollback/01_expand_down.sql`.
- [CAMPUS-110] Production sanity-тест с очисткой тестовых данных (Cleanup)
  и служебный эндпоинт удаления записей `sanity-bot`; стенд-тесты того же сценария для CI.
- [CAMPUS-110] Runbook `DatabaseConnectionPoolExhausted`, правила защиты ветки `main`, CI-
проверки `xunit-tests`, `db-migrations` и `k8s-manifests`.
- [CAMPUS-110] Манифесты Kubernetes: `deploy/k8s/sport-service.yaml` (RollingUpdate без
простоя,
  readiness/liveness `/health`, история ревизий для `kubectl rollout undo`) и манифест отката
  `deploy/k8s/rollback/sport-service-v1.5.3.yaml`.

### Fixed
- [CAMPUS-110] Двойное бронирование зала при одновременных запросах: проверка
  пересечения слотов теперь выполняется по диапазону, а не по строке.
- [CAMPUS-110] Слоты в формате `18.00 - 19.30` и `9:00-10:00` больше не
  отклоняются — приводятся к единому виду `HH:mm`.
- [CAMPUS-110] Некорректные слоты (конец раньше начала, вне часов работы
  07:00–23:00, короче 30 минут) отклоняются с ответом 400 и понятным сообщением.

### Deprecated
- [CAMPUS-110] Поле `timeSlot` в запросе и ответе API — используйте `timeRange`.
- [CAMPUS-110] Колонка БД `sport.bookings.time_slot`.
```

## 6. Вывод

Для модуля «Спортивный комплекс» (вариант 10, CAMPUS-110) выполнены все четыре блока. Цифровой след связывает задачу Jira с веткой, коммитами и тегами трассы `jira.issue` / `campus.entity_id`, а trace-id в логе совпадает с Trace ID в Jaeger. Ветку `main` защищают обязательные проверки CI (xUnit, миграции, манифесты Kubernetes), запрет push и force push, ревью двух коллег и Tech Lead. Изменения проходят через Pull Request. Релиз v1.6.0 оформлен Change Request с ночным окном обслуживания.

Откат приложения выполняется одной командой `kubectl rollout undo` (для резервного случая есть манифест отката на v1.5.3) и не требует

отката БД. Это обеспечивает паттерн Expand and Contract, работоспособность которого проверена на PostgreSQL 16. Sanity-тест записи на слот тренировки всегда очищает тестовые данные. Runbook, ADR и CHANGELOG хранятся в репозитории рядом с кодом (Docs-as-Code). Работа показала: регламенты и автоматизация надёжнее «ручных правок на сервере», потому что каждое изменение прослеживаемо и обратимо.